

Current research pipeline

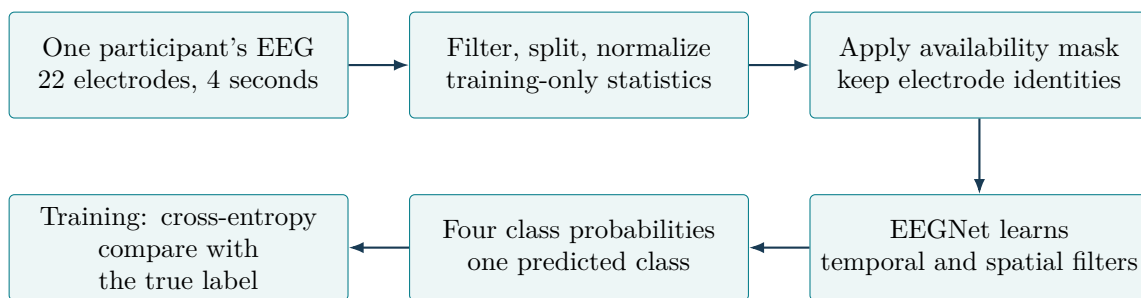
Dataset, EEG classification, and training losses

Scope: branch `research/baseline-accuracy`, source revision `4c27ec0`. Main example: `configs/baselines-reproducible.json`.

1 What are we trying to learn?

An EEG recording contains voltage measurements from electrodes on the scalp. For each four-second trial, the model predicts which movement the participant was *imagining*: left hand, right hand, feet, or tongue. We also ask how predictions change when some electrodes become unavailable.

Our immediate research direction is to understand a simple, reproducible EEGNet baseline before adding more architecture. The broader question is whether a tensor representation or completion method can improve robustness to electrode loss. Those are related questions, but they require separate comparisons.



At prediction time, the true label is not an input. Labels are needed to train the model and later to score its predictions.

Three designs currently discussed in the project

Design	What it does	Status in this guide
Current accuracy baseline	Raw masking, end-to-end EEGNet; also a covariance reference	Main explanation, pages 2–6
Earlier tensor pipeline	Frozen channel-local features, then tensor core/completion and attention	Explained separately on page 7
George's redesign	Raw-signal completion before EEGNet or Signal Transformer, both with MHA	Separate branch at <code>92b7fc2</code> ; page 7

The current baseline has no attention block and no Tucker module. Its convolutional feature extractor and classifier learn together from class labels. Tensor reconstruction is a separate objective used only in the tensor designs.

Reading order. Dataset and timing: page 2. Preprocessing and missing electrodes: page 3. Model architecture: page 4. Training loss: page 5. Checkpoint selection and evaluation: page 6. Tensor losses: page 7. Source map and references: page 8.

2 Dataset and the meaning of one trial

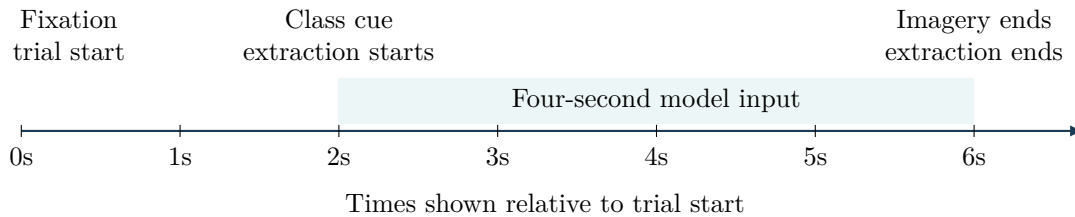
The recording collection

BCI Competition IV, dataset 2a contains nine participants, each recorded in two sessions on different days. Each session has six runs of 48 trials: 288 trials, nominally 72 per class. There are 22 EEG and three eye-movement (EOG) channels sampled at 250 Hz; our classifier uses only the EEG channels. The four imagery classes are left hand, right hand, feet, and tongue [1].

Item	Meaning in our default baseline
Input files	A01T.gdf through A09T.gdf; each participant is processed separately
Trial	1000 samples starting at a class cue; $1000/250 = 4$ seconds
One input	22×1000 numbers, or $22 \times 4 \times 250$ after reshaping
Target label	$y \in \{0, 1, 2, 3\}$: left hand, right hand, feet, tongue
Excluded trials	Artifact-marked, out-of-bounds, or nonfinite after filtering; counts are saved
Actual sample count	Depends on exclusions; do not assume all 288 trials survive

Cue time is different from trial-start time

In the recording paradigm, fixation starts at trial time 0, the movement cue appears at 2 seconds, and imagery continues until 6 seconds [1]. Our extraction begins at the *cue*, with zero additional offset.



The loader maps cue codes 769–772 to labels 0–3. It uses trial and run boundaries to avoid extracting across recordings. Each accepted trial keeps a stable ID so that its split membership and masks can be checked later.

Our default split answers a within-session question

For **each participant and seed**, accepted T-session trials are shuffled within each class and split approximately 60% training, 20% validation, and 20% test. Windows from a trial stay together; they are not independent split units. Integer rounding and artifact exclusions determine the exact counts. Training trials fit parameters. Validation trials choose the saved epoch. Test trials measure the selected model. This is a trial-level split within a participant’s T session, not a split by participant or recording run.

An E session is not the same as our default test partition. The current test partition comes from T. The separate cross-session configuration uses T for training/validation (80/20) and E exclusively for testing, with official E-session labels supplied explicitly. These protocols must be reported separately.

3 From a recording to a masked model input

Step 1: filter without mixing availability windows

The extracted four seconds are divided into four one-second windows. Each channel/window is filtered independently at 2–30 Hz using a Butterworth design with order argument 4 and forward–backward SOS filtering. The filter is fixed; it is not learned from class labels. It is zero-phase and offline, not causal. Filtering a window independently prevents an unavailable neighboring window from affecting an observed one before masking.

Step 2: normalize using training trials only

Let R_{ict} be the filtered value in trial i , channel c , sample t . For the training index set $\mathcal{T}$ and $T = 1000$:

$$\begin{aligned}\mu_c &= \frac{1}{|\mathcal{T}|T} \sum_{i \in \mathcal{T}} \sum_{t=1}^T R_{ict}, \\ \sigma_c &= \max \left(\sqrt{\frac{1}{|\mathcal{T}|T} \sum_{i \in \mathcal{T}} \sum_{t=1}^T (R_{ict} - \mu_c)^2}, 10^{-8} \right), \\ X_{ict} &= (R_{ict} - \mu_c) / \sigma_c.\end{aligned}$$

The same μ_c, σ_c are used on validation and test. They are not refitted per trial or per partition. Separate participant/seed tasks have their own training statistics.

Step 3: simulate known electrode loss

For a batch of B trials, reshape X to $[B, 22, 4, 250]$. The Boolean mask M has shape $[B, 22, 4]$. A value 1 means that the electrode is observed for the entire one-second window. The actual model input is

$$\tilde{X}_{bcp\tau} = \begin{cases} X_{bcp\tau}, & M_{bcp} = 1, \\ 0, & M_{bcp} = 0. \end{cases}$$

Implementation uses `torch.where` before any learned mixing. Multiplying by zero is unsafe if a hidden value is NaN. Zero after normalization represents the channel’s training mean; it does not mean a measured zero voltage.

	0–1 s	1–2 s	2–3 s	3–4 s
Electrode 1	teal	gray	teal	teal
Electrode 2	gray	teal	gray	gray
Electrode 3	teal	teal	teal	teal

Illustration: teal = observed, gray = hidden.
Only 3 of 22 electrodes are shown; time is relative to cue.

Full training uses all 22 electrodes. **Mixed training** chooses 22, 16, or 6 retained electrodes uniformly per trial each epoch. For 16/6, it chooses static or dynamic random loss equally. Electrode IDs never shift and no window may lose every electrode. The model’s convolutions may mix channels and windows *after* hidden values have been removed.

4 The current EEGNet classifier

EEGNet uses temporal, depthwise spatial, and separable convolutions [2]. This implementation is an architecture reproduction; its data split and preprocessing do not reproduce a paper’s complete benchmark protocol.

Follow the dimensions through the network

The four windows are joined back into 1000 time samples after masking. The extra dimension of size 1 below is an input feature-map axis. B is batch size.

Operation	Output shape	What it learns or does
Masked input	$B \times 1 \times 22 \times 1000$	Preserves the original channel order
Temporal convolution; BatchNorm	$B \times 8 \times 22 \times 1000$	Eight length-64 temporal filters, separately at each electrode
Depthwise spatial convolution; BatchNorm; ELU	$B \times 16 \times 1 \times 1000$	Two spatial filters per temporal feature, spanning all 22 electrodes
Average pooling by 4; dropout	$B \times 16 \times 1 \times 250$	Reduces temporal length; dropout rate 0.5
Separable convolution; BatchNorm; ELU	$B \times 16 \times 1 \times 250$	Length-16 depthwise temporal filters and pointwise feature mixing
Average pooling by 8; dropout	$B \times 16 \times 1 \times 31$	Keeps 31 ordered temporal bins; dropout rate 0.5
Flatten	$B \times 496$	$16 \cdot 31$ values, preserving bin order
Optional mask concatenation	$B \times 584$	Adds $22 \cdot 4 = 88$ availability flags
Linear classifier	$B \times 4$	Four scores, called logits

Reference EEGNet uses the 496 signal features alone. **Mask-conditioned EEGNet** also receives the 88 mask flags. Even the reference receives a mask for removing hidden samples; “reference” means it does not append those flags to the classifier. Neither version needs attention.

The declared comparison has four arms

Model	Training input	Evaluation coverage
EEGNet reference	Full	Full and electrode loss
Mask-conditioned EEGNet	Full	Full and electrode loss
Mask-conditioned EEGNet	Mixed	Full and electrode loss
Covariance + logistic regression	Full	Full only

There are $9 \text{ participants} \times 3 \text{ seeds} \times 4 \text{ arms} = 108$ fits. These are separate participant-specific models, not one network trained jointly on all participants.

Why include covariance? It supplies a simpler reference. Each filtered, unnormalized trial yields a 22×22 shrinkage covariance matrix. Its matrix logarithm is vectorized into 253 upper-triangle features, with off-diagonal entries weighted by $\sqrt{2}$. A training-fitted feature scaler and logistic classifier then predict the label. This is a log-Euclidean method; there is no neural encoder or tensor completion in this arm.

5 What loss trains the neural network?

From logits to probabilities

The network outputs four real numbers z_{ik} for trial i . Softmax turns them into probabilities:

$$p_{ik} = \frac{\exp(z_{ik})}{\sum_{j=0}^3 \exp(z_{ij})}, \quad \hat{y}_i = \arg \max_{k \in \{0,1,2,3\}} p_{ik}.$$

The predicted class is the largest probability. During training, the loss uses the probability of the *true* class, even when that class is not the largest.

Cross-entropy is the current baseline’s training objective

For a minibatch $\mathcal{B}$ with true labels y_i :

$$\mathcal{L}_{\text{CE}}(\theta) = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log p_{i,y_i}, \quad z_i = f_{\theta}(\tilde{X}_i, M_i).$$

For the reference model, M_i controls input masking but is not concatenated to the final features. The code passes *logits* directly to `cross_entropy`, which performs the stable log-softmax calculation. The baseline uses unweighted cross-entropy: no class weights, focal term, reconstruction loss, or explicit tensor penalty.

Numerical example. If the true class is left hand and the model assigns it probability 0.70, that trial’s loss is $-\log(0.70) \approx 0.357$. At probability 0.25 it is 1.386; at 0.05 it is 2.996. Confidently missing the true class produces a larger penalty. These are illustrative probabilities, not recorded results.

For a logit z_{ik} , the minibatch derivative is

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_{ik}} = \frac{p_{ik} - \mathbf{1}[k = y_i]}{|\mathcal{B}|}.$$

Backpropagation carries this signal through the linear classifier and all convolutions. Thus the features are learned for classification, not frozen in advance. Full and mixed training use the same loss; only input masks differ.

How parameters are updated

An epoch is one pass through training trials; each minibatch supplies one update.

Setting	Declared baseline value
Optimizer / minibatch	AdamW / 32 trials
Learning rate	Base 10^{-3} ; 10-epoch warmup then cosine decay
Weight decay	10^{-4} , applied by AdamW separately from the CE value
Gradient clipping	Global gradient norm at most 1
Weight constraints	Spatial filter max-norm 1.0; final classifier max-norm 0.25
Training budget	At most 250 epochs; early stopping eligible after 75
Patience	Stop after 50 epochs without a better validation selection score

Dropout and BatchNorm use training behavior during optimization. Validation uses evaluation mode. AdamW weight decay and max-norm constraints regularize parameters; they are not additional terms in the reported cross-entropy loss.

6 Training, model selection, and final evaluation

Three data roles and three different quantities

Partition	What happens	What it must not do
Training	Fit normalization statistics; fit model parameters by minimizing CE	Use validation/test statistics for fitting
Validation	Select the epoch by balanced accuracy; log-loss breaks ties	Backpropagate through validation examples
Test	Score the restored selected checkpoint	Select epochs, ranks, or architecture

The selected checkpoint is the epoch with the largest full-input validation balanced accuracy; a smaller validation log-loss wins a tie. Exact ties keep the earlier epoch. The final epoch is not necessarily the selected epoch.

$$\text{BA} = \frac{1}{4} \sum_{k=0}^3 \frac{\#\{i : y_i = k, \hat{y}_i = k\}}{\#\{i : y_i = k\}}, \quad \text{LogLoss} = -\frac{1}{N} \sum_i \log p_{i,y_i}.$$

Balanced accuracy gives each class equal weight, even when artifact removal changes class counts. Accuracy is the fraction of all correct trials. Neither accuracy nor balanced accuracy is the differentiable training loss.

One participant and seed, from start to finish

1. Load accepted trials, persist the split, and fit training normalization.
2. Initialize one arm with its declared seed. Train minibatches and compute validation scores after each epoch.
3. Save the best weights, constructor settings, normalization, class/channel order, and provenance. Restore and verify the saved checkpoint.
4. Freeze that selected model and score full and degraded input conditions. Run the other arms using matched splits, trial identities, and evaluation masks.

What electrode-loss evaluation measures

For neural arms, evaluate 22, 16, 11, and 6 retained electrodes. At each degraded count use static random, static spatial, dynamic random, and dynamic spatial loss. Static loss keeps one subset throughout a trial. Dynamic loss uses the four-window schedule A–B–A–A, so some electrodes disappear and return. Spatial patterns remove spatially structured subsets using fixed schematic electrode coordinates.

There are five repeats per degraded condition: $1 + 3 \cdot 4 \cdot 5 = 61$ mask conditions per partition and neural arm. The covariance reference has only full-input coverage. Eleven-channel and spatial-loss conditions are evaluation only. Banks are paired across arms, and nonfull channel subsets are allocated to separate training, validation, and test partitions.

Aggregation: average mask repeats within a condition, then seeds within each participant, then participants equally. Keep conditions distinct; report missing results rather than silently averaging a smaller cohort.

Optional robust selection is a different declared policy. The baseline package can instead average validation BA over full input and static/dynamic random loss at 16/6 electrodes. The reproducible baseline in this guide uses *full-input selection*. Do not confuse it with the completed tensor follow-up, which used its own declared robust validation rule.

7 Where tensors enter, and why their loss is separate

The earlier frozen-feature pipeline on our branch

Its stages are: train a channel/window-local encoder with a full-input multi-head attention (MHA) head using CE; select it on full validation; freeze it; fit training-only feature normalization and Tucker factors; then train separate attention classifiers using CE. Freezing disables encoder dropout and BatchNorm updates.

The encoder produces $H \in \mathbb{R}^{B \times 22 \times 4 \times 32}$. For one trial/window, write $H_p \in \mathbb{R}^{22 \times F}$, with $F = 32$. Tucker-2 uses

$$H_p \approx U G_p V^\top, \quad U \in \mathbb{R}^{22 \times r_c}, \quad V \in \mathbb{R}^{F \times r_f}, \quad G_p \in \mathbb{R}^{r_c \times r_f}.$$

Here U, V are shared factors; each trial/window has its own core G_p . Default legacy ranks are $r_c = r_f = 4$. The window axis is retained.

Unsupervised reconstruction and the masked ridge solve

For observed electrodes Ω_p , $n_p = |\Omega_p|$, and ridge $\lambda = 10^{-3}$:

$$\mathcal{J}_p(G_p; U, V) = \frac{1}{n_p F} \sum_{c \in \Omega_p} \sum_{f=1}^F [H_{pcf} - (U G_p V^\top)_{cf}]^2 + \lambda \|G_p\|_F^2.$$

Factor fitting averages this objective over training trials/windows. It alternates core solves with Adam updates of U, V , normalizing each factor column to unit length. It does not use class labels. Fitted factors then become frozen buffers. For each new masked input, solve for its core using observed values only:

$$(U_{\Omega_p}^\top U_{\Omega_p}) G_p (V^\top V) + (n_p F \lambda) G_p = U_{\Omega_p}^\top H_{p, \Omega_p} V.$$

The factor $n_p F$ follows from averaging squared error over observed entries. Positive ridge makes the conditional core unique; it does not guarantee recovery of missing signal or preservation of class information.

Core classification feeds G_p to a classifier. **Completion** keeps observed entries exactly and replaces only missing ones by $U G_p V^\top$. Both then train their classifier using CE with factors fixed. **There is no jointly optimized CE-plus-reconstruction loss** in this staged design. Better reconstruction need not mean better classification.

George's separate raw-completion design

At origin/main revision 92b7fc2, the tensor input is normalized *raw* windows, so $F = 250$, $r_c = 4$, and $r_f = 16$. Training-only factor fitting precedes an end-to-end EEGNet or Signal Transformer with built-in MHA. Both backbones train on full inputs using CE. Completion is bypassed for full input, and fills missing raw values only during degraded evaluation. This is a different experiment from the frozen-feature comparison.

A possible future controlled test: use one selected, strong EEGNet checkpoint and compare zero-fill with training-fitted raw completion under the same masks. This is a proposed next experiment, not the current baseline implementation or an established accuracy improvement.

8 How to connect this guide to the repository

Files that define the current baseline

File	Responsibility
<code>configs/baselines-reproducible.json</code>	Declared participants, arms, split, and training settings
<code>inm/data.py</code>	GDF cues, exclusions, and window-local filtering
<code>agfl/datasets/splits.py</code>	Persisted trial partitions
<code>inm/baselines/data.py</code>	Training normalization and prepared trial data
<code>inm/availability.py</code>	Electrode identities and paired availability masks
<code>inm/baselines/eegnet.py</code>	Raw masking and the EEGNet architecture
<code>agfl/optimization.py</code>	Cross-entropy implementation
<code>inm/baselines/training.py</code>	AdamW, validation selection, checkpoint saving/reloading
<code>inm/baselines/covariance.py</code>	Log-covariance and logistic reference
<code>inm/baselines/reporting.py</code>	Coverage checks and hierarchical aggregation
<code>inm/tensor_attention.py</code>	Earlier Tucker objective and masked core solve

The covariance arm minimizes a multinomial logistic classification objective with L2 regularization ($C = 1$), using L-BFGS with at most 1000 iterations. It does not use the neural epoch schedule. Its feature scaler and classifier are fitted on training data; its covariance calculation is performed per trial.

Inspect a plan without launching training

From the repository root:

```
python -m inm.baselines \
    --config configs/baselines-reproducible.json --plan
```

This prints 27 participant/seed tasks and 108 fits. By contrast, `python run.py --plan` describes the earlier experiment: 27 tasks and 378 classifier fits. They are different entry points and different protocols.

What remains a research question

Our current priority is to understand training and validation behavior of the spatial EEGNet baseline before adding complexity. A high training score and lower validation score should prompt checks of generalization, trial alignment, and preprocessing. They do not by themselves identify the cause.

This document explains source behavior, not a new experiment. No training was run to produce it. Existing empirical reviews are in `docs/baselines/`; the branch comparison is `docs/baselines/branch-review.md`. The older `agfl_concept.pdf` remains a detailed companion for the frozen-feature tensor mathematics. Changed source, packages, or experiment settings require a new result directory with its own provenance.

References

- [1] C. Brunner et al. *BCI Competition 2008, Graz data set A* (BCI Competition IV, dataset 2a). Official recording and task description. https://www.bbci.de/competition/iv/desc_2a.pdf.
- [2] V. J. Lawhern et al. *EEGNet: A Compact Convolutional Network for EEG-based Brain-Computer Interfaces*. 2018. <https://arxiv.org/abs/1611.08024>.